

1.Objectif du test :

L'objectif est de valider le bon fonctionnement d'un petit robot piloté via un protocole de communication ASCII.

Le robot peut avancer ou reculer, et il accepte trois types de commandes principales :

/AR move ds : déplacement du robot d'une distance **ds** (elle peut être positive ou négative).

/AR get <param> : lecture d'une donnée (**pos** ou **speed** dans notre cas).

/AR set <param> <valeur> : modification d'une donnée (dans notre cas, uniquement **speed** entre 1 et 10).

AR : adresse du robot

Chaque commande envoyée reçoit une réponse ASCII du type :

@AR OK <valeur>

ou

@AR RJ <erreur>

2.Détails du système :

Élément	Description
Adresse (AR)	Adresse du robot
Commande	move, get, ou set
Arguments	Distance ou paramètre selon la commande
Réponse	Toujours au format @AR OK (cas succès) ou @AR RJ (cas erreur)
Variables principales	pos : position actuelle (un entier signé) speed : vitesse du robot (de 1 à 10)

Le protocole de communication est un texte **pur (ASCII)** et peut être piloté depuis un PC via port série ou simulation.

3.Plan de tests proposés

Les tests proposés sont regroupés en plusieurs catégories : des tests fonctionnels de base, bornes, robustesse et protocole, et comportement dynamique.

Test fonctionnels de base :

N°	Test	Objectif	Étapes	Résultat attendu
T1	Lecture de la position initiale	Vérifier que get pos retourne une valeur valide	/01 get pos	@01 OK <entier>
T2	Lecture de la vitesse par défaut	Vérifier que get speed retourne une valeur dans [1..10]	/01 get speed	@01 OK <valeur 1..10>
T3	Déplacement avant	Vérifier que la position augmente correctement	Lire pos0 → /01 move 1000 → relire pos	pos_final = pos0 + 1000
T4	Déplacement arrière	Vérifier que la position diminue correctement	Lire pos1 → /01 move -500 → relire pos	pos_final = pos1 - 500
T5	Déplacement nul	Vérifier que move 0 n'a aucun effet	Lire posA → /01 move 0 → relire posB	posB = posA

Test sur la variable speed :

N°	Test	Objectif	Étapes	Résultat attendu
T1	Bornes acceptées	Vérifier que les valeurs limites sont acceptées	/01 set speed 1, puis get speed /01 set speed 10, puis get speed	@01 OK 1 @01 OK 10
T2	Hors bornes rejetées	Vérifier le rejet des vitesses invalides	/01 set speed 0, /01 set speed 11, /01 set speed -1	@01 RJ RANGE ou erreur équivalente
T3	Type invalide	Vérifier la robustesse au parsing	/01 set speed abc, /01 set speed 3.5	@01 RJ BADVAL
T4	Persistance	Vérifier que la vitesse reste la même après un move	/01 set speed 7, puis move 1000, puis get speed	@01 OK 7

Tests protocole et robustesse

N°	Test	Objectif	Étapes	Résultat attendu
T1	Commande inconnue	Vérifier le rejet des commandes invalides	/01 tourne 10	@01 RJ UNKNOWN
T2	Arguments manquants	Vérifier la détection d'erreurs de syntaxe	/01 move, /01 set speed	@01 RJ BADARGS / BADVAL / BADFMT
T3	Adresse incorrecte	Vérifier l'isolation entre robots	/02 get pos (seul 01 connecté)	Aucune réponse
T4	Espaces multiples	Vérifier la tolérance des séparateurs	/01 get pos	@01 OK <valeur> (le simulateur développé tolère les espaces multiples)
T5	Mauvaise casse	Vérifier la sensibilité aux majuscules	GET pos, Set speed 5	@01 OK <valeur> (simulateur développé tolérant)

				à la casse — accepte les deux)
--	--	--	--	-----------------------------------

Tests dynamiques (le comportement temporel)

N°	Test	Objectif	Étapes	Résultat attendu
T1	Test de vitesse réelle	Vérifier que le robot va plus vite quand speed augmente	Mesurer le temps d'un move 2000 pour speed = 2, 5 et 10	$t(2) > t(5) > t(10)$
T2	Test d'évolution de la position	Vérifier que la position change de manière régulière pendant le déplacement (sans à-coups ni retour en arrière)	Poller get pos pendant un move (Lire plusieurs fois get pos pendant un move en cours)	valeurs croissantes jusqu'à la cible
T3	Test de stabilité après déplacement	Vérifier qu'une fois la cible atteinte, la position ne varie plus	Lire get pos plusieurs fois après la fin du move	même valeur sur N lectures

4.Proposition d'automatisation des tests

4.1 Outils & environnement:

Langage utilisé : Python 3

Framework de test : **pytest**

Environnement d'exécution : macOS (MacBook Air)

Robot simulé : **SimSerial** (simulateur conforme au protocole ASCII)

Client haut-niveau : **RobotClient** (gère les commandes /AR cmd args ↔ @AR OK/RJ val)

4.2 Test choisi

L'énoncé demandait d'automatiser un test du plan de validation.

Dans le cadre de ce travail, j'ai choisi d'aller plus loin en **automatisant l'ensemble des tests définis** dans la section 3, couvrant ainsi **toutes les fonctionnalités et tous les scénarios du protocole ASCII** du robot.

L'automatisation a été réalisée en **Python** à l'aide du framework **Pytest**, sur un **simulateur logiciel** respectant le protocole de communication ASCII décrit dans la spécification. Cette approche permet de valider le comportement du robot sans matériel physique, tout en reproduisant fidèlement les échanges texte du protocole.

4.3 Structure technique de la solution

src/

simulator.py # Simule le robot (position, vitesse, déplacements, réponses ASCII)

robot_client.py # Interface Python pour envoyer les commandes et lire les réponses

tests/

test_functional.py # Tests fonctionnels de base

test_speed_rules.py # Tests sur la variable speed

test_protocol.py # Tests de robustesse protocole

test_dynamics.py # Tests dynamiques : position et stabilité

test_speed_timing.py # Test dynamique : influence de la vitesse

test_with_hardware.py Test sur robot réelle

Chaque fichier de test correspond directement à une catégorie du plan de validation. Tous les tests sont indépendants et s'exécutent sur une instance de robot simulé propre grâce à une fonction utilitaire `new_bot()`.

4.4 Méthode d'automatisation

Chaque test traduit un scénario du tableau du plan de test :

- Les commandes ASCII `/01 move`, `/01 get`, `/01 set` sont envoyées par le client Python.
- Les réponses du robot simulé sont lues au format `@01 OK ...` ou `@01 RJ ....`
- Les valeurs reçues sont analysées et comparées à la valeur attendue via des **assertions automatiques**.

4.5 Résultats d'exécution

L'ensemble des tests automatisés ont été exécutés localement sur macOS avec succès.

Fichier	Tests	Résultat
test_functional.py	5 tests fonctionnels de base	5 passed
test_speed_rules.py	4 tests sur la variable speed	4 passed
test_protocol.py	5 tests protocole et robustesse	5 passed
test_dynamics.py	2 tests dynamiques (monotonie, stabilité)	2 passed
test_speed_timing.py	1 test dynamique (vitesse réelle)	1 passed
Total	17 tests automatisés	Tous validés

4.6 Points techniques importants

- Synchronisation temporelle assurée via `wait_until()` pour attendre la fin réelle du mouvement avant la vérification.
- Simulation fidèle du protocole ASCII : chaque commande `/AR cmd args` génère une réponse `@AR OK/RJ val`.
- Tolérance d'erreurs : les rejets (RJ) sont validés pour plusieurs codes possibles (BADVAL, BADARGS, BADFMT) selon la syntaxe.
- Indépendance : chaque test crée une nouvelle instance du robot, garantissant l'absence d'interférences entre cas.

4.7 Conclusion sur l'automatisation

Bien que la consigne initiale prévoyait l'automatisation d'un test, l'ensemble du plan de validation a été automatisé et exécuté avec succès. Cette démarche démontre une approche complète de test et validation, incluant :

- la vérification fonctionnelle des commandes,
- la validation des bornes et erreurs,
- et l'évaluation dynamique du comportement du robot.

Les résultats (tous "passed") confirment la conformité du simulateur et du protocole aux exigences spécifiées.